

Paper 1: O2-IMS-to-Operator-CRD Runtime Architecture for Closed-Loop O-RAN Day-2 Remediation

David Kypuros

Red Hat

dkypuros@redhat.com

31 May 2026

Abstract

Paper 1 in this five-paper series describes the O2-IMS-to-operator-CRD runtime architecture for closed-loop O-RAN Day-2 remediation. The pattern composes standards and implementation contract layers: O-RAN O2 General Aspects and Principles (O2 GA&P) for the O-Cloud resource model; O-RAN O2 Infrastructure Management Services (O2 IMS) as the SMO-to-O-Cloud infrastructure handoff; Kubernetes-native operator CRD surfaces (Metal3 BareMetal Operator (BMO) HostFirmwareComponents, Machine Config Operator (MCO) MachineConfig, and Kernel Module Management (KMM) Module resources) as the delivery substrate; TM Forum TMF688/TMF921-shaped audit and intent envelopes; and IEEE 1588 / O-RAN WG6 Cloud Notifications as the timing-domain observation contract. We claim the novelty lies in the O2-IMS-to-operator-CRD bridge: a deterministic routing rule classifies infrastructure faults against the O2 GA&P O-Cloud resource model, hands the proposal through an O2-IMS-shaped handoff in v0, and represents delivery on existing Kubernetes operator contracts before any large language model is consulted. The contribution extends a prior multivendor diagnostic substrate by closing the loop from fault to audited remediation through a schema gate, a five-subcheck policy guardrail, and human co-authorization. The runtime is open-source and shipped with a verify gate that produces a binary pass/fail any reviewer can reproduce. We acknowledge that v0 runtime is stub-driven: it demonstrates an O2 IMS-shaped stub boundary and CRD-shaped fixtures, not live O2 IMS interop, live reconcile loops, or production wiring.

1 Introduction

Day-2 operations on a multivendor Cloud RAN deployment span a wide functional surface: timing drift on a worker node, firmware regressions on accelerator NICs, host service interference with the PTP hardware clock, driver swaps for fixed-function offloads, configuration churn from upstream SMO intents. Each of these is observable, each has an audited path to remediation, and yet the bridges between diagnosis and execution are uneven across vendors and standards bodies.

Prior agentic-AI work for RAN automation has clus-

tered around two failure modes. The first stops at the SMO boundary: an agent converges on a root cause and emits an intent, but the intent never crosses into a concrete delivery contract. The second invents a parallel controller plane that side-steps O-RAN's own resource layering, which is operationally fragile and politically expensive in a multivendor ecosystem. A prior multivendor diagnostic publication established a fault-localization substrate for Cloud RAN troubleshooting [9] but left the loop open: the diagnostic agents converge on a root cause and emit an artifact, while the actual remediation path remains undefined.

This paper closes the loop. We describe a harness pattern that takes the diagnostic artifact as input, classifies it against a deterministic taxonomy grounded in the O-RAN O-Cloud resource model [19, 25], routes the resulting proposal through an O2-IMS-shaped adapter boundary [26] toward Kubernetes-native CRD contracts (Metal3 BMO HostFirmwareComponents [16] and MCO MachineConfig resources [29], with KMM Module resources [10] as a third delivery path), and emits a TM Forum TMF688-shaped audit event [32] with a harness-specific reversibility profile before any apply is authorized. The four pieces (observation loop, cognitive middle, routing decision, sandbox plus operator seam) are described in order. We name the architectural novelty the **O2-IMS-to-operator-CRD bridge**: O2 GA&P [25] supplies the O-Cloud resource-layer model; O2 IMS [26] supplies the SMO-to-O-Cloud infrastructure handoff shape; Metal3 BMO [16] supplies the bare-metal firmware and provisioning CRD path; and MCO [29] supplies the host-configuration CRD path. The taxonomy subtypes are harness-defined mappings into the O2 GA&P Physical Resource, Logical Resource, and Managed O-Cloud Service classes; they are not proposed extensions to the O-RAN specification. The claim is not that any one interface is new. The claim is that their simultaneous composition lets a single **RemediationProposal** move from the runtime routing layer, through a standards-shaped O-RAN boundary, toward existing Kubernetes-native reconciler contracts without inventing a parallel controller plane. KMM is an extensibility witness within the same infrastructure route rather than part of the named four-element bridge.

The remainder of the paper is organized as follows. Section 2 establishes the standards and project context for each interface composed. Section 3 describes the

architecture in three logical pieces. Section 4 summarizes the prototype, the five committed walkthrough scenarios, and the verify gate. Section 5 discusses positioning against adjacent projects and the standards-track ask. Section 6 concludes.

2 Background

O-RAN architecture and the O-Cloud surface.

The O-RAN Alliance Working Group 1 architecture description [19] and Working Group 6 O2 General Aspects and Principles (O2 GA&P) [25] jointly define the O-Cloud resource model and the SMO-to-O-Cloud interface family. The O2 IMS specification [26] carries the infrastructure provisioning request and response envelopes; O2 DMS [24] carries deployment management. This work targets O2 IMS exclusively: the harness handles infrastructure remediation, not deployment lifecycle. The reference O2 IMS implementation cited throughout is Red Hat’s open-source O-Cloud Manager [28]. The Decoupled SMO Architecture technical report [21] introduces the SMO Service (SMOS) abstraction, into which a closed-loop remediation harness fits naturally as a candidate first-class SMOS.

Delivery substrate. Below the O2 IMS interface, three Kubernetes-native operators carry the actual change: the Machine Config Operator (MCO) reconciles MachineConfig resources for host configuration [29], the Kernel Module Management Operator (KMM) reconciles Module resources for driver swaps [10], and the Metal3 BareMetal Operator (Metal3 BMO) reconciles BareMetalHost and HostFirmwareComponents resources for bare-metal firmware and provisioning [16,17]. Out-of-band firmware writes pass through the host BMC via DMTF Redfish [5].

Timing and observation. The PTP stack on each worker uses linuxptp [14] to discipline the PTP Hardware Clock (PHC) against an upstream Grandmaster, conforming to IEEE 1588-2019 [7] and the ITU-T G.8275.1 telecom profile [8]. The Red Hat PTP Operator [6] orchestrates linuxptp on the cluster, and the cloud-event-proxy sidecar [30] publishes state transitions as CloudEvents [4] carrying the O-RAN WG6 Cloud Notifications payload [20]. The harness consumes that notification envelope as the inbound fault contract.

Audit and intent. TM Forum TMF688 [32] provides the event-management envelope used as the audit sink; the ReversibilityProfile carried here is harness-specific. TMF921 [33] provides the intent management envelope used for the dual-route notification path from the harness to the partner SMO; the `dispatch_result` field is likewise harness-specific. The SID information framework [31] grounds the conceptual model that maps O-RAN resource classes to TM Forum entity types.

3 System Architecture

The architecture has three logical pieces plus an explicit governance layer: the left loop (observation), the cognitive middle inside the Agent Harness, the right loop

(routing, delivery, and traceability), and the NOC operator plus peer reviewers above the harness. The left loop is the observation loop under discussion here: bare-metal timing signals from the NIC PHC and linuxptp stack are carried by the PTP Operator and cloud-event-proxy into a CloudEvents-shaped `FaultPayload`. The Killswitch is a feature of the core harness guardrail path, and the Digital Twin is where validation takes place. Figure 1 shows the macro layout.

3.1 Left loop: observation

Telemetry originates in the timing domain. Each Cloud RAN worker runs a PTP Hardware Clock disciplined by linuxptp daemons (`ptp4l` and `phc2sys`) [14], conforming to IEEE 1588-2019 [7] and the G.8275.1 telecom profile [8]. The PTP Operator [6] runs the daemons and the cloud-event-proxy sidecar [30] watches their state transitions. When the daemon state crosses a threshold (SYNCHRONIZED to HOLDOVER, master offset spike, monotonic PHC drift), the sidecar emits a CloudEvent [4] with the O-RAN WG6 Cloud Notifications payload [20].

The harness consumes this envelope as a `FaultPayload` (a JSON Schema draft-07 contract carried at `harness/schemas/FaultPayload.json`). The repository’s scenario `D_phc_drift_hw_only` demonstrates the canonical divergence pattern that gives the harness its diagnostic edge: `ptp4l` reports SYNCHRONIZED (software view of the timing chain is healthy) while `phc2sys` reports monotonic frequency drift in parts-per-billion and the NIC reports rising `tx_hwtstamp_timeouts` (hardware view is not healthy). A purely software-layer observer would miss this; the harness reads both.

3.2 Cognitive middle

The Agentic Gateway terminates the CloudEvents stream and presents vendor-private tooling to the agent runtime under controlled MCP interfaces [1,2,15]. The v0 repository declares four MCP tool surfaces behind the gateway: `mcp-platform` (linuxptp daemons, host services, driver versions, NIC counters), `mcp-ran` (cell sync, PM counters, RAN parameters), `mcp-hardware` (NIC PHC introspection, BMC Redfish, CPU performance counters), and `mcp-ocloud` (the O-Cloud Manager inventory and alarms). A production deployment would run these as FastMCP servers with mTLS, OAuth2, and IP allowlists; the v0 lab treats the surfaces as local or stubbed and trusts localhost.

Three domain agents (Platform, RAN, Hardware) consume the MCP surface and converge on a root cause artifact (RCA). The cognitive middle’s deterministic core is the Remediation Router. The router consults a 20-entry taxonomy declared in `harness/taxonomy.yaml`. Twelve infrastructure entries are grounded in O2 GA&P Physical Resource, Logical Resource, or Managed O-Cloud Service classes; five service-layer entries are routed upward through TMF921; and three ambiguous entries require disambiguation before execution. The taxon-

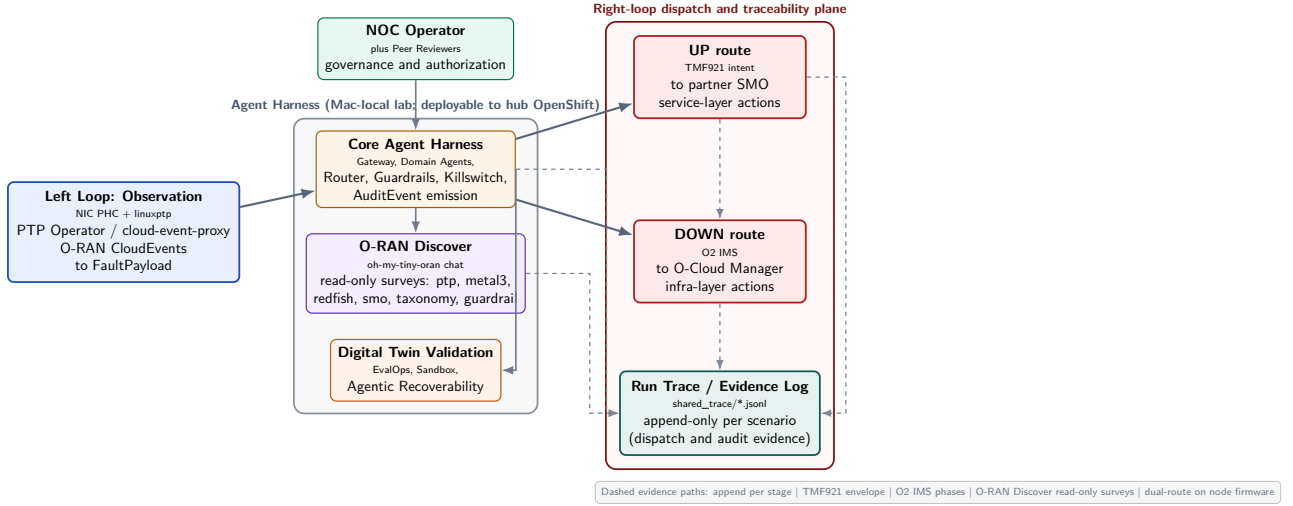


Figure 1: Seven-component macro architecture. The left loop is the observation loop: NIC PHC and linuxptp state are watched by the PTP Operator and cloud-event-proxy, emitted as O-RAN CloudEvents, and ingested as the harness `FaultPayload`. The NOC operator and peer reviewers sit above the harness as the governance and authorization layer. The Agent Harness contains the Agentic Gateway, MCP-backed domain agents, deterministic Router, guardrails, Killswitch, audit emission, O-RAN Discover read-only operator inspection surface, and Digital Twin validation substrate. The right side packages the service UP route, infrastructure DOWN route, and append-only run trace as one dispatch and traceability plane, so TMF921/O2 IMS handoffs and their evidence trail remain visually coupled.

omy carries an explicit grounding disclaimer in its first eighteen lines: the subtypes are interpretive mappings within, or deliberate layer-boundary separations from, the normative resource classes in O2 GA&P Figure 3.4-1, not extensions to the spec.

The deterministic taxonomy lookup fires first; an LLM is consulted only when the result is the `ambiguous` class. This ordering is enforced in code and proven by the test suite. The repository’s `contribution-1-routing-rule.yaml` declares the rule, the runtime at `harness/runtime/router.py` implements it, and the test `test_route_ambiguous_with_llm_mode_live` demonstrates that the LLM seam is wired through without becoming load-bearing on the routing decision. The live LLM test is support evidence for the seam, not part of the deterministic gate proof.

The LLM seam is provider-neutral: `completion()` can target Anthropic Claude, the OpenAI API, or an OpenAI-compatible on-premises vLLM endpoint such as Red Hat OpenShift AI [3]. The MacBook dashboard can use `ANTHROPIC_API_KEY`, an OpenAI key, or an OpenAI-compatible local endpoint; the deterministic verify gate and the five committed scenarios do not depend on any hosted model.

3.3 Right loop: routing and dual-route delivery

The routing decision splits along the O-RAN resource-layer boundary. **Service-layer** faults (cell re-home, RAN parameter changes, slice intent updates, vDU software version rollouts) belong to the partner SMO. The harness does not execute them; it emits a TMF921-shaped companion intent [33] and observes the harness-

specific dispatch result. This is the service route.

Infrastructure-layer faults go through the O2 IMS interface [25, 26] into the O-Cloud Manager [28], which dispatches into one of three reconcilers: MCO [29] for host configuration changes, KMM [10] for driver swaps, and Metal3 BMO [16] for node firmware. Firmware writes pass through the host BMC via DMTF Redfish UpdateService.SimpleUpdate [5]. This is the infrastructure route. In the v0 implementation, the infrastructure route is a real Python function call from the router into an O2 IMS-shaped stub boundary that returns a shape-correct `o2ims_dispatch` envelope and points at committed CRD-shaped fixtures; the envelope is attached to the audit so the operator sees the intended O2 IMS hop as a row in the record, not as live interop or a synthesized path string.

The dual-route pattern fires both branches simultaneously when an infrastructure action carries inherent service-layer impact. Scenario `E_nic_firmware_update` is the teaching case: a NIC firmware update via Metal3 BMO takes the host offline for a reboot window, so the harness emits a TMF921-shaped companion intent to the partner SMO in parallel with the O2 IMS infrastructure-route apply. The partner SMO’s response (acceptance or rejection) lands on the audit’s harness-specific `dispatch_result` field before operator co-authorization. Scenario `E_with_smo_reject` demonstrates the rejection path: same infrastructure-route verdict, SMO rejects the maintenance window because neighboring cells are at 92 percent capacity. The operator sees both outcomes and is the reconciliation point.

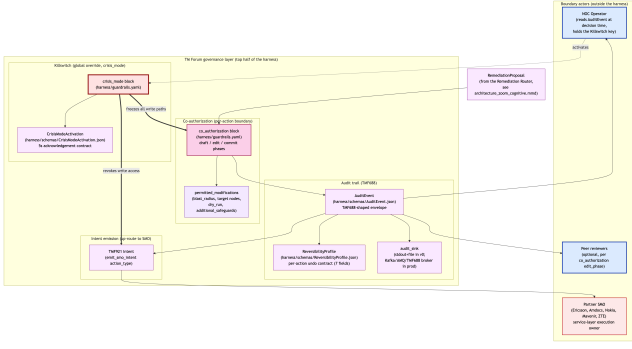


Figure 2: Governance layer zoom. The TMF688-shaped audit event carries the proposal, the guardrail result, the sandbox verdict, the optional companion intent with its harness-specific dispatch result, and the seven-field ReversibilityProfile. The co-authorization contract sits between draft and commit. The Killswitch (crisis_mode) is the global override.

3.4 Sandbox gate and reversibility

Before any apply, the proposal traverses three deterministic control points: a JSON Schema gate, a five-subcheck policy guardrail declared at `harness/guardrails.yaml` and executed at `harness/runtime/guardrail.py`, and human co-authorization. The policy guardrail fires its subchecks in order: `crisis_mode` global override, `sandbox apply_allowed`, `action_allowlist`, `blast_radius caps` (one node, one site, four cells in v0), and `requiresHumanApproval`. The v0 runtime evaluates the sandbox subcheck against committed verdict fixtures; a production deployment would source that verdict from a same-topology mirrored cluster or a simulated linuxptp plus NIC driver stack.

Every TMF688-shaped audit event carries a populated harness-specific `ReversibilityProfile` with seven fields: `rollback intent`, `blast-radius-if-reverse`, `rebuild timeline`, `replication history`, `validation history`, `risk profile burn-down`, and `confidence in reversibility`. The intent is that the operator has the rollback contract at decision time, not after-the-fact. Figure 2 shows the governance layer in detail.

3.5 Killswitch as global override

A `crisis_mode` flag in `guardrails.yaml` is the global override. When activated by a NOC supervisor in response to a macro event (storms, security incidents, regional outages), all write paths revoke, EvalOps confidence pins to zero, and telemetry routes to a manual queue. The Reversibility Profile rewinds one applied action; the Killswitch freezes all of them. The two compose.

3.6 Operator-facing discovery surface

The MacBook lab exposes the cognitive middle through `oh-my-tiny-oran`, a small OMC-shaped chat harness scoped to the read-only `oran-discover` bundle at `omc-skills/oran-discover/`. The chat service inlines

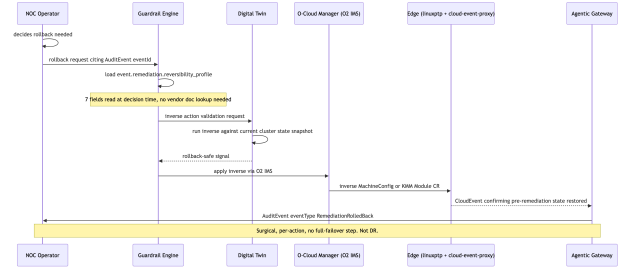


Figure 3: Sequence diagram for a single closed-loop remediation cycle. PTP CloudEvent arrives at the gateway, traverses the runtime router to a RemediationProposal, passes the schema gate and five-subcheck policy guardrail, lands on the operator's human co-authorization seat, and emits a TMF688-shaped audit event with the harness-specific reversibility profile attached.

the invoked skill markdown into the turn and permits only bounded reads: HTTP GET calls against lab wrappers and reads from committed repository files. The seven operator skills are therefore cognitive scaffolding, not remediation machinery. Six probes inspect one surface each, and `/oran-discover:plan` composes them as a pre-flight walk before the operator considers any write-capable workflow.

The reviewer-facing right-loop demo deliberately keeps two surfaces separate. The `/commands` and `/oran-discover:*` paths are read-only inspection surfaces for PTP, Metal3, Redfish, SMO, taxonomy, and guardrail state. The `harness-walker/run/{scenario}` path is a separate deterministic sidecar that returns the `AuditEvent`; the chat assistant may inspect or explain that event, but it does not become the router, guardrail, or approval authority.

4 Prototype and Walkthroughs

The repository ships five end-to-end fixture scenarios under `scenarios/`: `A_fw_lldp_agent` (a host service interfering with PTP, routed to MCO), `A_prime_ice_driver` (an outdated NIC driver causing PHC drift, routed to KMM), `D_phc_drift_hw_only` (the hardware-only divergence demonstrating the diagnostic edge), `E_nic_firmware_update` (the dual-route firmware case with accepted companion intent), and `E_with_smo_reject` (the same firmware path with a rejected companion intent). Each scenario carries a `fault_payload.json` input, a remediation YAML, and a committed `audit_event.json` fixture that the verify gate replays against the actual walker output.

The empirically exercised MacBook/Sheldon walkthrough makes three right-loop branches visible without expanding the v0 interop claim. In `D_phc_drift_hw_only`, the taxonomy selects `host_driver` and the down route is O2 IMS to the O-Cloud KMM driver path. In `E_nic_firmware_update`, the taxonomy selects `node_firmware`; the harness shows the O2 IMS down route toward Metal3/Redfish and the TMF921 companion intent up route toward the

Skill	Primary harness surface	Read path summarized for the operator
<code>/oran-discover:ptp</code>	Left-loop observation	PTP sync state, PHC drift, linuxptp state, and CloudEvents / O-RAN Cloud Notifications wrapper output.
<code>/oran-discover:metal3</code>	Infrastructure route	Metal3 BMO BareMetalHost and HostFirmwareComponents state behind the O2 IMS shaped handoff.
<code>/oran-discover:redfish</code>	Hardware and BMC substrate	Redfish BMC task state, cross-checked against the Metal3 firmware path rather than exposed as a direct write path.
<code>/oran-discover:smo</code>	Service route	TMF921-shaped companion intent state and the partner-SMO response surface used by dual-route cases.
<code>/oran-discover:taxonomy</code>	Cognitive middle	The 20-entry routing taxonomy, including infrastructure, service, and ambiguous layer-boundary classes.
<code>/oran-discover:guardrail</code>	Governance overlay	The deterministic policy state: <code>allowlist</code> , <code>blast-radius caps</code> , <code>requiresHumanApproval</code> , and <code>crisis_mode</code> .
<code>/oran-discover:plan</code>	Pre-flight orchestration	A read-only walk across the six probes so the operator can inspect evidence, routing, and policy before authorization.

Table 1: The `oran-discover` operator skills. They expose the left loop, cognitive middle, right-loop service and infrastructure state, and governance overlay as read-only inspection surfaces.

SMO. In `E_with_smo_reject`, the same infrastructure route remains shape-correct while the SMO companion dispatch is rejected before human co-authorization, leaving the operator as the reconciliation point. In all three branches the endpoints are stubs, but the routing decision, guardrail result, sandbox verdict, `AuditEvent`, and chat/sidecar wiring are real harness behavior.

The v0 runtime pipeline (walker, router, sandbox-fixture evaluation, guardrail) is deterministic. A reviewer can clone the repository, install PyYAML and jsonschema, and run `python3 scripts/verify.py`. With jsonschema installed, the gate completes ten checks in under five seconds: citation header completeness on every authored YAML and JSON, JSON and YAML parse, traceability-table bidirectional completeness, an em-dash audit, a leakage guard, README structural sanity, file counts, JSON Schema validation of every scenario fault payload plus the embedded remediation and reversibility-profile subobjects, and end-to-end walker replay against committed audit fixtures on eight material comparisons (seven named fields plus the reversibility profile confidence sub-field). In reduced environments where jsonschema is absent, check 9 is reported as skipped and the reduced gate reports the executed checks.

5 Discussion

The O2-IMS-to-operator-CRD bridge. The architecture’s central claim is that a single `RemediationProposal` travels through the O2 GA&P resource model and an O2 IMS contract surface [25, 26], then is represented at an existing Kubernetes-native operator CRD reconciler such as Metal3 BMO or MCO [16, 29] without a custom controller plane in between. In v0, this is demonstrated as fixture replay over a shape-correct O2 IMS stub plus committed CRD-shaped fixtures, not live O2 IMS interop. The conjunction of O2 GA&P, O2 IMS, Metal3 BMO, and MCO is not a re-derivation of prior architectures; existing agentic-RAN work either stops at the SMO boundary without a delivery story, or proposes a parallel controller plane. The harness pattern composes existing contracts.

Comparison to ONAP, Nephio, OSM. ONAP

[12], Nephio [11], and Open Source MANO [13] are orchestration platforms that manage network function lifecycle at scale. The harness pattern is closed-loop remediation: fault-driven cognition that emits a proposal, gates it, audits it, and observes the operator’s commit. The two compose. Nephio can deliver the standing topology; the harness handles post-deployment Day-2 anomalies on top of that topology.

Positioning against the Decoupled SMO Architecture. The closed-loop harness fits the SMOS abstraction described in WG1 TR R004 [21]. It exposes a TMF688-shaped audit surface and a TMF921-shaped intent surface as a candidate SMOS-compatible interface shape (interop testing against an SMOS-conformant catalogue is a v1 work item). A reasonable standards-track ask is for an nGRG output that names a Closed-Loop Remediation SMOS as a first-class element of the Decoupled SMO catalogue, and a TM Forum Implementation Guide companion that documents two harness-unique extensions as candidate API additions: a `dispatch_result` field on the TMF921 companion intent envelope, and the seven-field `ReversibilityProfile` attached to every TMF688-shaped audit event.

Layer-boundary discipline. The harness is explicit about what it does not target. O-RAN R1 [27] is the SMO-internal service-exposure interface for rApps; the harness’s MCP gateway is internal agent scaffolding, not R1. A1 [18] targets Near-RT RIC behavior (slice SLA, traffic steering); the harness targets the infrastructure layer below. E2 [22] governs Near-RT RIC interactions with O-CU and O-DU; the harness does not consume E2 directly. O1 [23] governs SMO-to-managed-element OAM; the harness operates below the managed-element boundary because the O-Cloud is platform, not managed element. These disclaimers are not modesty; they preserve the composability claim.

Honest-scope acknowledgements. The v0 runtime is stub-driven for the five committed scenarios. The platform stubs at `5G_O-RAN_SIM/oam/` emit shape-correct envelopes without crossing real network boundaries. The five-subcheck policy guardrail enforces v0 absolute limits (one node, one site, four cells); a production deployment would wire to a policy-as-code system (OPA or Kyverno) and source limits from a service catalog. Multi-cluster scale is not demonstrated. The co-authorization edit-phase (operator amending the pro-

posal before commit instead of approving or rejecting it) is the v1 target; v0 enforces a boolean approval gate.

6 Conclusion

This paper has described a closed-loop Day-2 remediation harness pattern for O-RAN Cloud RAN deployments. The pattern composes pre-existing standardized and implementation contracts (O2 GA&P, O-RAN O2 IMS, Kubernetes-native operator CRDs, TM Forum TMF688 and TMF921, IEEE 1588 plus O-RAN Cloud Notifications) under a deterministic routing rule grounded in the O-RAN WG6 O-Cloud resource model. The O2-IMS-to-operator-CRD bridge, O2 GA&P plus O2 IMS plus Metal3 BMO plus MCO, is the architectural novelty, with KMM as an additional co-delivery path. The runtime is open-source, exercised end-to-end by a fixture-driven verify gate, and ships with five walkthrough scenarios including a dual-route firmware case with both accepted and SMO-rejected variants. v0 limits are declared in code and in this paper: fixture replay demonstrates an O2 IMS-shaped stub boundary and CRD-shaped fixtures, not live O2 IMS interop, live reconcile loops, or production interop.

Acknowledgements

This work extends a prior multivendor diagnostic substrate published as an Ericsson Technology Blog and co-authored with S. Kiani Mehr, N. Korati Prasanna, M. Vazquez Cebrian, S. Srikanthan, and P. Venkatesh [9]. Killswitch design feedback was received from operator stakeholders. Apache-2.0 licensed source code: <https://github.com/dkypuros/oran-agent-harness>.

References

- [1] Anthropic. Model context protocol python SDK. <https://github.com/modelcontextprotocol/python-sdk>, 2026. Accessed 14 May 2026.
- [2] Anthropic. Model context protocol specification. <https://modelcontextprotocol.io>, 2026. Accessed 14 May 2026.
- [3] BerriAI. LiteLLM: unified interface to call 100+ LLM APIs using the OpenAI input/output format. <https://github.com/BerriAI/litellm>, 2026. Accessed 14 May 2026.
- [4] Cloud Native Computing Foundation. CloudEvents version 1.0 specification: a vendor-neutral specification for describing event data. <https://cloudevents.io>, 2026. Accessed 14 May 2026.
- [5] Distributed Management Task Force. Redfish specification (DMTF DSP0266). <https://www.dmtf.org/standards/redfish>, 2026. Out-of-band management of servers and infrastructure. Accessed 14 May 2026.
- [6] Red Hat. PTP operator for OpenShift documentation. https://docs.redhat.com/en/documentation/openshift_container_platform, 2026. Accessed 14 May 2026.
- [7] Institute of Electrical and Electronics Engineers. IEEE Std 1588-2019: IEEE standard for a precision clock synchronization protocol for networked measurement and control systems. <https://standards.ieee.org/ieee/1588/6825/>, 2019.
- [8] International Telecommunication Union, Telecommunication Standardization Sector. ITU-T G.8275.1: Precision time protocol telecom profile for phase / time synchronization with full timing support from the network. <https://www.itu.int/rec/T-REC-G.8275.1>, 2022.
- [9] S. Kiani Mehr, N. Korati Prasanna, D. Ky-puros, M. Vazquez Cebrian, S. Srikanthan, and P. Venkatesh. Multivendor agentic AI solution for cloud RAN troubleshooting. Ericsson Technology Blog. <https://www.ericsson.com/en/blog/2026/5/multivendor-agentic-ai-solution-for-cloud-ran>, 2026. Published 11 May 2026. Accessed 14 May 2026.
- [10] Kubernetes Special Interest Groups. Kernel module management operator (KMM). <https://github.com/kubernetes-sigs/kernel-module-management>, 2026. Accessed 14 May 2026.
- [11] Linux Foundation Networking. Nephio: Cloud-native network automation. <https://nephio.org>, 2026. Accessed 14 May 2026.
- [12] Linux Foundation Networking. Open network automation platform (ONAP). <https://www.onap.org>, 2026. Accessed 14 May 2026.
- [13] Linux Foundation Networking. Open source MANO (OSM). <https://osm.etsi.org>, 2026. Accessed 14 May 2026.
- [14] linuxptp project. ptp4l(8) manual page: PTP boundary / ordinary clock daemon for Linux. <https://linuxptp.sourceforge.net>, 2026. Accessed 14 May 2026.
- [15] J. Lowin. FastMCP: a pythonic framework for building model context protocol servers and clients. <https://github.com/jlowin/fastmcp>, 2026. Accessed 14 May 2026.
- [16] Metal3 project. Baremetal operator: BareMetal-Host, HostFirmwareSettings, HostFirmwareComponents Custom Resource Definitions. <https://github.com/metal3-io/baremetal-operator>, 2026. Accessed 14 May 2026.
- [17] Metal3 project. Metal3: Bare metal host provisioning for Kubernetes. <https://metal3.io>, 2026. Accessed 14 May 2026.

- [18] O-RAN Alliance. O-RAN A1 interface: General aspects and principles and application protocol, working group 2 (O-RAN.WG2.TS.A1GAP-R005-v05.03 and O-RAN.WG2.TS.A1AP-R005-v06.00). <https://www.o-ran.org/specifications>, 2026. Accessed 30 May 2026.
- [19] O-RAN Alliance. O-RAN architecture description, working group 1 specification. <https://www.o-ran.org/specifications>, 2026. Accessed 14 May 2026.
- [20] O-RAN Alliance. O-RAN cloud notifications specification, working group 6 (O-RAN.WG6.Cloud-Notifications). <https://www.o-ran.org/specifications>, 2026. Accessed 14 May 2026.
- [21] O-RAN Alliance. O-RAN decoupled SMO architecture, working group 1 technical report (O-RAN.WG1.TR.Decoupled-SMO-Architecture-R004-v03.00). <https://www.o-ran.org/specifications>, 2026. Accessed 30 May 2026. Source for SMOS terminology.
- [22] O-RAN Alliance. O-RAN E2 interface family: General aspects and principles, application protocol, and service models, working group 3 (E2GAP, E2AP, E2SM-KPM, E2SM-RC, E2SM-LLC). <https://www.o-ran.org/specifications>, 2026. Accessed 30 May 2026.
- [23] O-RAN Alliance. O-RAN O1 interface specification, working group 10 (O-RAN.WG10.TS.O1-Interface.0-R005-v18.00). <https://www.o-ran.org/specifications>, 2026. Accessed 30 May 2026.
- [24] O-RAN Alliance. O-RAN O2 DMS interface specification (deployment management services), O-RAN.WG6.O2DMS-Interface. <https://www.o-ran.org/specifications>, 2026. Working Group 6 specification. Accessed 14 May 2026.
- [25] O-RAN Alliance. O-RAN O2 general aspects and principles, version 01.02 (O-RAN.WG6.O2-GANP-v01.02). <https://www.o-ran.org/specifications>, 2026. Working Group 6 specification, with successor in R005 release train (O-RAN.WG6.TS.O2-GA&P-R005-v10.00). Accessed 14 May 2026.
- [26] O-RAN Alliance. O-RAN O2 IMS interface specification (infrastructure management services), O-RAN.WG6.O2IMS-Interface. <https://www.o-ran.org/specifications>, 2026. Working Group 6 specification. Accessed 14 May 2026.
- [27] O-RAN Alliance. O-RAN R1 interface: General aspects and principles and application protocols for R1 services, working group 2 (O-RAN.WG2.TS.R1GAP-R005-v13.00 and O-RAN.WG2.TS.R1AP-R005-v10.00). <https://www.o-ran.org/specifications>, 2026. Accessed 30 May 2026.
- [28] OpenShift KNI. oran-o2ims: O-Cloud manager. <https://github.com/openshift-kni/oran-o2ims>, 2026. Accessed 14 May 2026.
- [29] OpenShift project. Machine config operator: declarative configuration of Red Hat Enterprise Linux CoreOS. <https://github.com/openshift/machine-config-operator>, 2026. Accessed 14 May 2026.
- [30] Red Hat Cloud Native Events project. cloud-event-proxy: PTP notification publisher emitting O-RAN-compliant CloudEvents from linuxptp state changes. <https://github.com/redhat-cne/cloud-event-proxy>, 2026. Accessed 14 May 2026.
- [31] TM Forum. GB922 information framework (SID) concepts and principles. <https://www.tmforum.org>, 2026. Accessed 14 May 2026.
- [32] TM Forum. TMF688 event management API user guide and REST specification. <https://www.tmforum.org/oda/open-apis/table/tmf688>, 2026. Accessed 14 May 2026.
- [33] TM Forum. TMF921 intent management API user guide and REST specification. <https://www.tmforum.org/oda/open-apis/table/tmf921>, 2026. Accessed 14 May 2026.